

**LAPORAN PRAKTIKUM ALGORITMA
DAN PEMROGRAMAN 2**

**MODUL 17
SKEMA PEMROSESAN SEKUENSIAL**



Disusun oleh:

Rafi Azis Faozan

109082500069

S1IF-13-02

Dosen:

Wahyu Andi Saputra, S. Pd., M.Eng

**PROGRAM STUDI S1 INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO**

2025

LATIHAN KELAS – GUIDED

1. Soal 1

Source Code

```
package main

import "fmt"

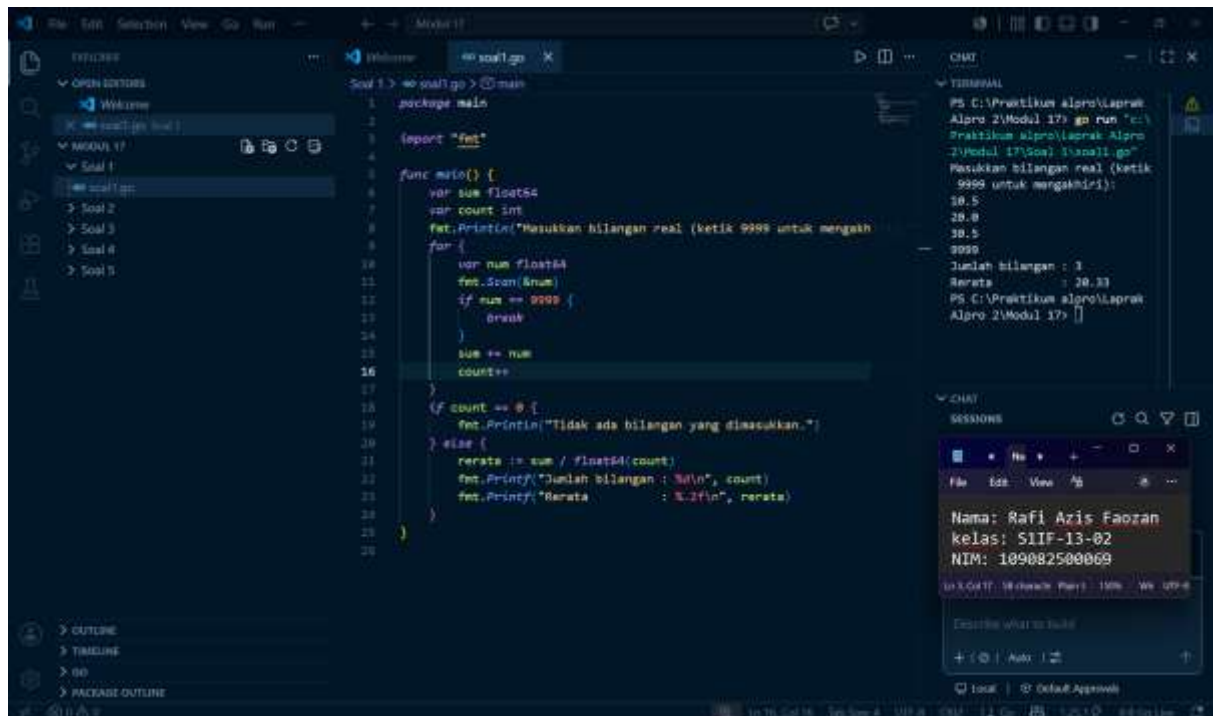
func main() {
    var sum float64
    var count int

    fmt.Println("Masukkan bilangan real (ketik 9999
untuk mengakhiri):")

    for {
        var num float64
        fmt.Scan(&num)
        if num == 9999 {
            break
        }
        sum += num
        count++
    }

    if count == 0 {
        fmt.Println("Tidak ada bilangan yang
dimasukkan.")
    } else {
        rerata := sum / float64(count)
        fmt.Printf("Jumlah bilangan : %d\n", count)
        fmt.Printf("Rerata          : %.2f\n", rerata)
    }
}
```

Screenshoot program



Deskripsi program

Program ini menggunakan bahasa Go yang berfungsi untuk menerima sejumlah bilangan real sebagai input secara berulang hingga pengguna memasukkan angka 9999 sebagai penanda akhir data. Setelah marker ditemukan, program menghitung dan menampilkan nilai rata-rata dari seluruh bilangan yang telah dimasukkan sebelumnya.

2. Soal 2

Source Code

```

package main

import "fmt"

func main() {
    var x string
    fmt.Print("Masukkan string x: ")
    fmt.Scan(&x)
    var n int
    fmt.Print("Masukkan jumlah data (n): ")
    fmt.Scan(&n)
    data := make([]string, n)

```

```

    fmt.Println("Masukkan", n, "buah string:")

    for i := 0; i < n; i++ {
        fmt.Printf("  Data ke-%d: ", i+1)
        fmt.Scan(&data[i])
    }

    ditemukan := false

    for _, s := range data {
        if s == x {
            ditemukan = true
            break
        }
    }

    fmt.Println("\n=== HASIL ===")

    if ditemukan {
        fmt.Printf("a. String \"%s\" ADA dalam kumpulan
data.\n", x)
    } else {
        fmt.Printf("a. String \"%s\" TIDAK ADA dalam
kumpulan data.\n", x)
    }

    fmt.Printf("b. Posisi string \"%s\" ditemukan: ", x)
    posisi := []int{}

    for i, s := range data {
        if s == x {
            posisi = append(posisi, i+1)
        }
    }

    if len(posisi) == 0 {
        fmt.Println("tidak ditemukan.")
    } else {
        fmt.Println(posisi)
    }

    jumlah := len(posisi)

```

```

    fmt.Printf("c. Jumlah string \"%s\" dalam kumpulan
data: %d\n", x, jumlah)

    if jumlah >= 2 {

        fmt.Printf("d. Terdapat SEDIKITNYA DUA string
\"%s\" dalam kumpulan data.\n", x)

    } else {

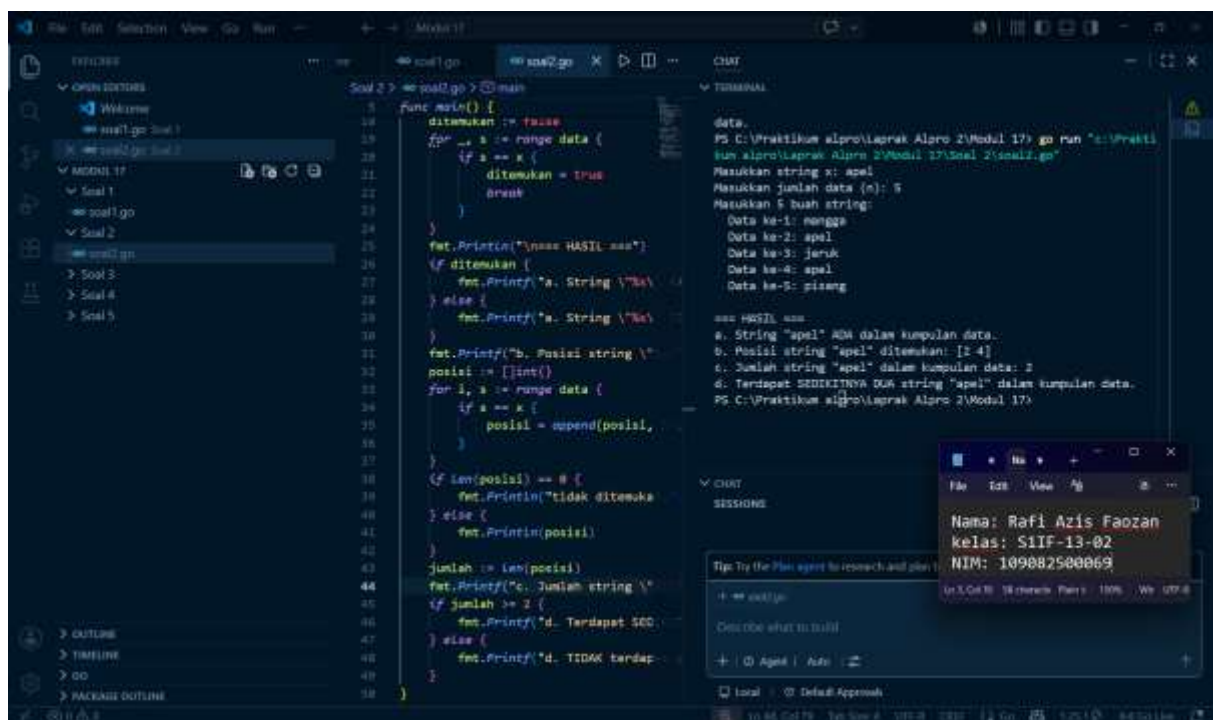
        fmt.Printf("d. TIDAK terdapat sedikitnya dua
string \"%s\" dalam kumpulan data.\n", x)

    }

}

```

Screenshoot program



Deskripsi program

Program ini berjalan menggunakan bahasa Go yang menerima sebuah string `x` sebagai kata kunci pencarian, diikuti bilangan `n` dan `n` buah string sebagai kumpulan data. Program kemudian melakukan pencarian untuk menjawab empat pertanyaan: apakah string `x` ditemukan dalam kumpulan data, pada posisi mana saja string `x` muncul, berapa kali string `x` ditemukan, serta apakah string `x` muncul setidaknya dua kali dalam kumpulan data tersebut.

3. Soal 3

Source Code

```
package main
```

```

import (
    "fmt"
    "math/rand"
)

func main() {
    var n int
    fmt.Print("Masukkan banyaknya tetesan air hujan: ")
    fmt.Scan(&n)
    countA, countB, countC, countD := 0, 0, 0, 0
    for i := 0; i < n; i++ {
        x := rand.Float64()
        y := rand.Float64()
        if x < 0.5 && y < 0.5 {
            countA++
        } else if x >= 0.5 && y < 0.5 {
            countB++
        } else if x >= 0.5 && y >= 0.5 {
            countC++
        } else {
            countD++
        }
    }

    const perTetes = 0.0001

    fmt.Printf("Curah hujan daerah A: %.4f\n", float64(countA)*perTetes)

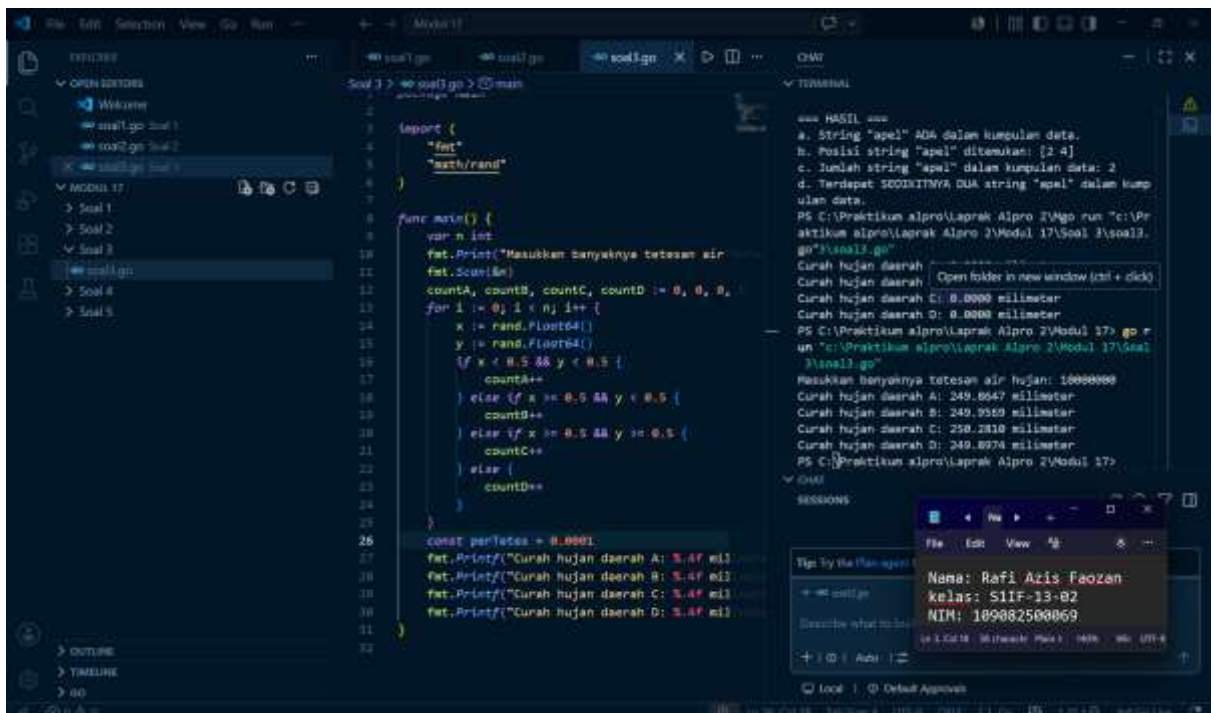
    fmt.Printf("Curah hujan daerah B: %.4f\n", float64(countB)*perTetes)

    fmt.Printf("Curah hujan daerah C: %.4f\n", float64(countC)*perTetes)

    fmt.Printf("Curah hujan daerah D: %.4f\n", float64(countD)*perTetes)
}

```

Screenshoot program



Deskripsi program

Program ini berjalan menggunakan bahasa Go yang mensimulasikan pengukuran curah hujan pada empat daerah berbentuk persegi (A, B, C, D) yang membagi bidang koordinat (0,0) hingga (1,1) menjadi empat kuadran sama besar. Program menerima input berupa jumlah tetesan hujan, lalu membangkitkan titik acak (x,y) menggunakan rand.Float64() untuk setiap tetesan. Setiap titik kemudian diklasifikasikan ke salah satu daerah berdasarkan posisinya, dan hasil akhirnya ditampilkan sebagai curah hujan dalam satuan milimeter dengan konversi 1 tetesan = 0.0001 milimeter.

4. Soal 4

Source code

```
package main

import (
    "fmt"
    "math"

)

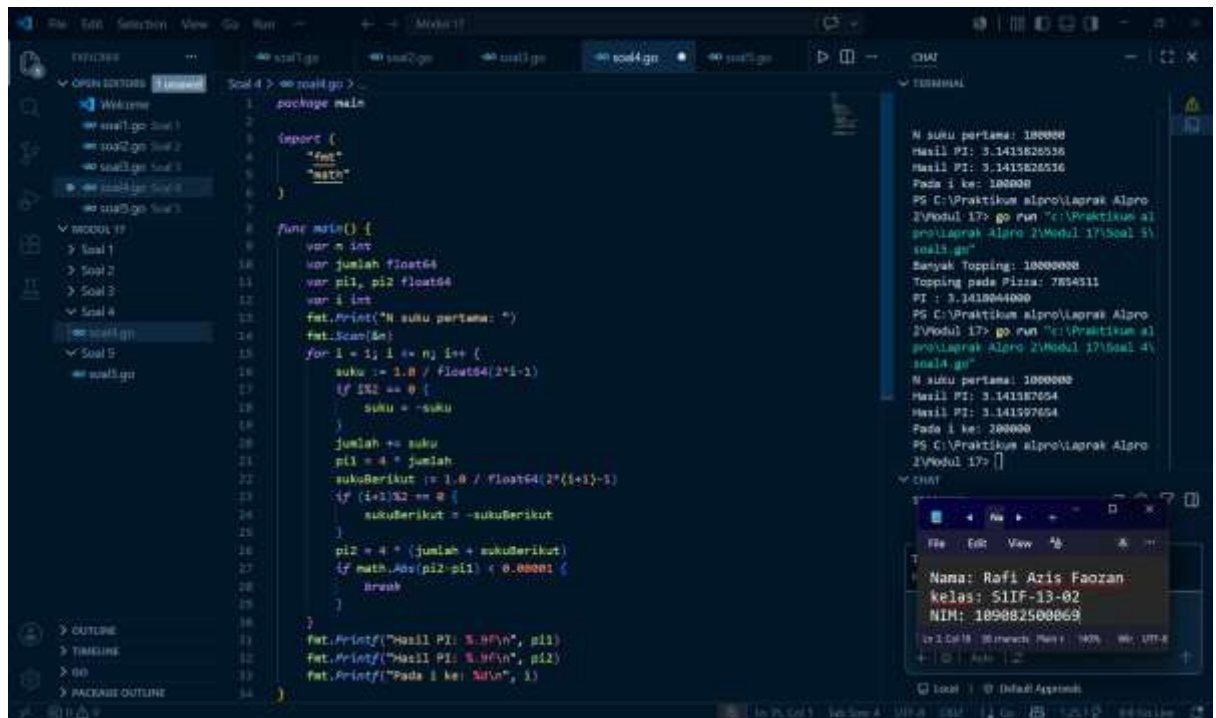
func main() {
    var n int
    var jumlah float64
    var pi1, pi2 float64
```

```

var i int
fmt.Print("N suku pertama: ")
fmt.Scan(&n)
for i = 1; i <= n; i++ {
    suku := 1.0 / float64(2*i-1)
    if i%2 == 0 {
        suku = -suku
    }
    jumlah += suku
    pi1 = 4 * jumlah
    sukuBerikut := 1.0 / float64(2*(i+1)-1)
    if (i+1)%2 == 0 {
        sukuBerikut = -sukuBerikut
    }
    pi2 = 4 * (jumlah + sukuBerikut)
    if math.Abs(pi2-pi1) < 0.00001 {
        break
    }
}
fmt.Printf("Hasil PI: %.9f\n", pi1)
fmt.Printf("Hasil PI: %.9f\n", pi2)
fmt.Printf("Pada i ke: %d\n", i)
}

```

Screenshoot program



Deskripsi program

Program ini berjalan menggunakan bahasa Go yg mana menghitung nilai konstanta π menggunakan formula Leibniz, yaitu deret harmonik berganti tanda $1 - 1/3 + 1/5 - 1/7 + \dots = \pi/4$. Program terdiri dari dua bagian. Bagian pertama menerima input N dan menjumlahkan N suku pertama deret tersebut, lalu mengalikan hasilnya dengan 4 untuk mendapatkan nilai π . Bagian kedua memodifikasi proses penjumlahan dengan menyimpan dua nilai jumlah deret yang bersebelahan, S_i dan S_{i+1} , kemudian berhenti secara otomatis apabila selisih nilai π dari kedua suku tersebut tidak lebih dari 0.00001, dan menampilkan posisi suku ke-i saat kondisi berhenti terpenuhi.

5. Soal 5

Source code

```

package main

import (
    "fmt"
    "math"
    "math/rand"
)

func main() {
    var n int

```

```

fmt.Print("Banyak Topping: ")

fmt.Scan(&n)

xc, yc, r := 0.5, 0.5, 0.5

count := 0

for i := 0; i < n; i++ {
    x := rand.Float64()
    y := rand.Float64()

    if math.Pow(x-xc, 2)+math.Pow(y-yc, 2) <= r*r {
        count++
    }
}

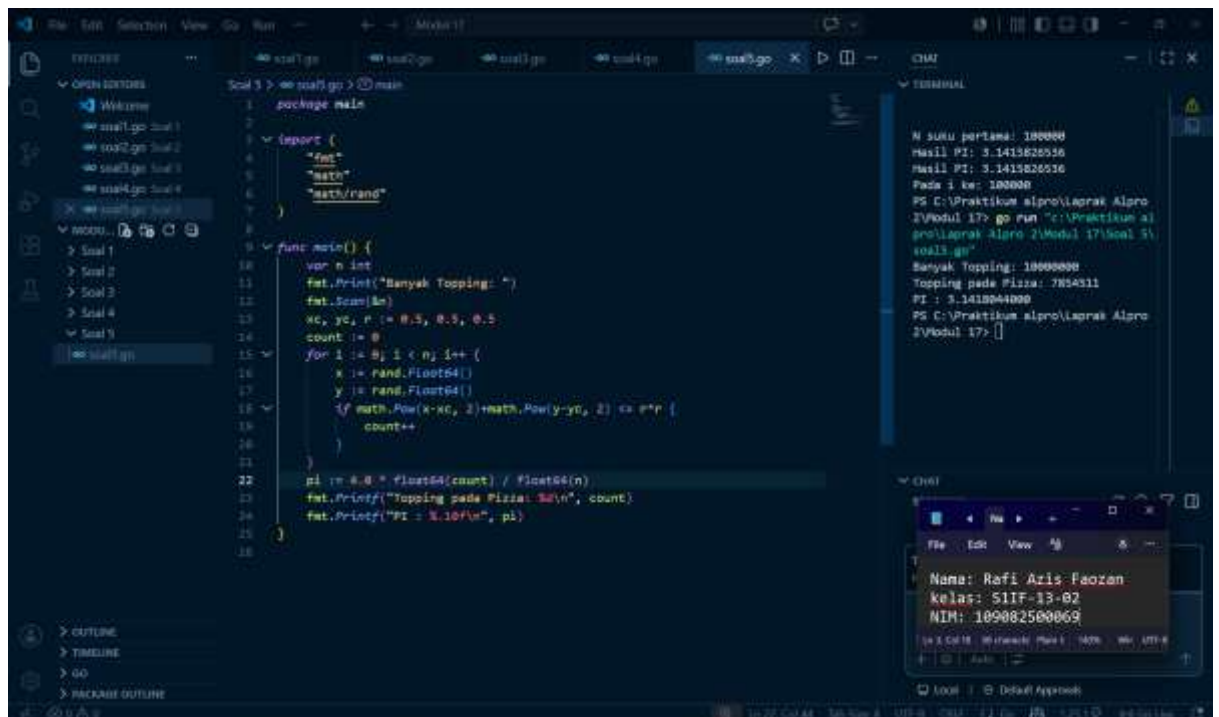
pi := 4.0 * float64(count) / float64(n)

fmt.Printf("Topping pada Pizza: %d\n", count)

fmt.Printf("PI : %.10f\n", pi)
}

```

Screenshoot program



Deskripsi program

Program ini berjalan menggunakan bahasa Go yang mensimulasikan penaburan topping pizza menggunakan metode Monte Carlo untuk menghitung nilai konstanta π . Program menerima input berupa jumlah topping yang ditaburkan, lalu membangkitkan titik acak (x, y) menggunakan `rand.Float64()` untuk setiap topping. Setiap titik kemudian diperiksa apakah jatuh di dalam lingkaran pizza berdasarkan persamaan $(x - 0.5)^2 + (y - 0.5)^2 \leq 0.25$, dengan pusat pizza di (0.5, 0.5) dan jari-jari 0.5. Program menampilkan banyaknya topping yang jatuh tepat di atas pizza, serta mengestimasi nilai π berdasarkan perbandingan luas pizza terhadap luas wadah persegi dengan rumus $\pi \approx 4 \times (\text{topping di pizza} / \text{total topping})$.